Punteros en C

Ejercicios · Alumnado

Resuelve los diez ejercicios en orden. Utiliza #include <stdio.h> y una función int main(void) con return 0;. Coloca las funciones auxiliares y las estructuras antes de main. No es necesario reservar memoria dinámica.

Nombre: _____ Fecha: _____

1. Una variable y su puntero

Parte de int numero = 10;. Declara un puntero p que apunte a numero. Muestra el valor de numero, su dirección, la dirección almacenada en p y el valor al que apunta p. ¿Por qué deben coincidir las dos direcciones?

2. Modificar una variable

Declara int temperatura = 20; y un puntero que apunte a ella. Muestra el valor inicial, cambia la temperatura a 25 mediante el puntero y muestra el valor final.

3. Predecir el resultado

Sin ejecutarlo, indica qué mostrará el siguiente programa. Explica qué variable modifica *p = 8;, qué cambia con p = &b; y qué variable modifica la última asignación.

```
int a = 5;
int b = 12;
int *p = &a;
*p = 8;
p = &b;
*p = *p + 3;
printf("a = %d\n", a);
printf("b = %d\n", b);
printf("*p = %d\n", *p);
```

4. Completar el código

Completa los huecos para que el programa muestre Cantidad: 30.

```
int cantidad = 15;
int *p = ________;
________ = 30;
printf("Cantidad: %d\n", cantidad);
```

5. Detectar y corregir errores

Corrige el siguiente fragmento. Explica qué debe almacenarse en p y qué tipo de dato espera %d.

```
int numero = 7;
int *p;
p = numero;
*p = 14;
printf("Numero: %d\n", &numero);
```

6. Incrementar mediante una función

Crea void incrementar(int *valor); para aumentar en una unidad la variable señalada. En main declara int contador = 0;, llama tres veces a la función y muestra el resultado. Salida esperada: Contador: 3.

7. Intercambiar dos valores

Crea void intercambiar(int *a, int *b);. En main declara primero = 10 y segundo = 20, ambos int. Muestra sus valores antes y después de intercambiarlos mediante la función. Pista: utiliza una variable auxiliar.

8. Recorrer un array con un puntero

Declara int memorias[] = {4, 8, 16, 8, 32};. Recorre sus cinco elementos con un puntero. Dentro del bucle accede mediante *(p + i). Explica la diferencia entre p + i y *(p + i).

9. Sumar los elementos de un array

Crea int sumar(const int *datos, int cantidad);. Debe devolver la suma de los elementos. Prueba con int valores[] = {3, 6, 9, 12};. Salida esperada: Suma: 30. ¿Por qué hay que indicar la cantidad de elementos?

10. Modificar una estructura

Declara la estructura indicada antes de main. Crea void ampliarRAM(struct Equipo *equipo, int aumento); para sumar aumento a su RAM. En main crea struct Equipo equipo = {"PC01", 8};, añade 8 GB mediante la función y muestra el nombre y la RAM final: PC01 y 16 GB. Pista: utiliza ->.

```
struct Equipo
{
    char nombre[20];
    int ram;
};
```